Copias de seguridad en PostgreSQL: realización y restauración

Administración y diseño de bases de datos · Grado en Ingeniería Informática · ULL
Francisco Javier Cañas González

1. Objetivo y entorno

Se realiza la **actividad práctica 2**: hacer una copia de seguridad de una base de datos y restaurarla, tanto en una base de datos nueva como sobrescribiendo la original. Se ha trabajado por consola (alternativa permitida en el enunciado), con las mismas herramientas que utiliza pgAdmin por debajo: `pg_dump` y `pg_restore`.

La base de datos es `clientesdb` (tablas `clientes` y `pedidos`), desplegada con Docker Compose a partir del repositorio de la asignatura (`postgresql-backups/src`, imagen `postgres:16.3`). Solo se levanta el servicio `pgdb`, ya que el servicio `pgbackup` pertenece a la actividad 1. Los comandos se ejecutan **dentro del contenedor** con `docker compose exec`, lo que garantiza que las versiones de `pg_dump`, `pg_restore` y del servidor coinciden.

2. Pasos realizados

Paso 1. Despliegue y estado inicial

Se levanta el servicio y se comprueba que está en ejecución. A continuación se consulta el número de filas de cada tabla y un **resumen MD5** del contenido. Este resumen es la clave de la verificación: si tras restaurar el MD5 coincide, los datos son idénticos fila a fila, no solo "parecidos".

```
docker compose up -d pgdb
docker compose ps
docker compose exec pgdb psql -U admin -d clientesdb -c "\dt"
docker compose exec pgdb psql -U admin -d clientesdb -c "SELECT 'clientes' AS tabla, count(*),
        md5(string_agg(c::text, ',' ORDER BY id)) FROM clientes c UNION ALL SELECT 'pedidos', count(*),
        md5(string_agg(p::text, ',' ORDER BY id)) FROM pedidos p;"
```

```
$ docker compose up -d pgdb
 Container src-pgdb-1 Running
$ docker compose ps
NAME         IMAGE          COMMAND                  SERVICE   CREATED          STATUS          PORTS
src-pgdb-1   postgres:16.3   "docker-entrypoint.s…"   pgdb      27 seconds ago   Up 26 seconds   5432/tcp
$ docker compose exec pgdb psql -U admin -d clientesdb -c "\dt"
         List of relations
 Schema |   Name   | Type  | Owner
--------+----------+-------+-------
 public | clientes | table | admin
 public | pedidos  | table | admin
(2 rows)

$ docker compose exec pgdb psql -U admin -d clientesdb -c "SELECT 'clientes' AS tabla, count(*),
    md5(string_agg(c::text, ',' ORDER BY id)) FROM clientes c UNION ALL SELECT 'pedidos', count(*),
    md5(string_agg(p::text, ',' ORDER BY id)) FROM pedidos p;"
  tabla   | count |               md5
----------+-------+----------------------------------
 clientes |     3 | 6fa9339d883070e9a704739cd835dcee
 pedidos  |     3 | 0126617ce5584df79682981be43f6da7
(2 rows)
```

Despliegue del servicio y estado inicial de clientesdb

Paso 2. Realización de la copia de seguridad

Se generan dos copias con `pg_dump` para poder compararlas:

- **Formato personalizado (`-Fc`)**: binario, comprimido, y el único que permite una restauración selectiva con `pg_restore`.
- **Formato SQL plano (por defecto)**: texto legible con las sentencias `CREATE TABLE` y `COPY`, que se restaura con `psql`.

```
docker compose exec pgdb pg_dump -U admin -d clientesdb -Fc -f /tmp/clientesdb.dump
docker compose exec pgdb pg_dump -U admin -d clientesdb -f /tmp/clientesdb.sql
docker compose exec pgdb ls -lh /tmp/clientesdb.dump /tmp/clientesdb.sql
docker compose exec pgdb pg_restore -l /tmp/clientesdb.dump
```

El último comando lista el **índice (TOC)** de la copia personalizada: tablas, secuencias, datos y restricciones que contiene. Es la prueba de que la copia es válida antes de necesitarla. `pg_dump` obtiene una instantánea consistente gracias a MVCC, sin bloquear a los usuarios que estén leyendo o escribiendo durante la copia.

Además, se copia el fichero al equipo anfitrión, porque una copia que solo vive dentro del mismo contenedor que la base de datos no protege frente a un fallo de ese contenedor:

```
docker compose cp pgdb:/tmp/clientesdb.dump ./clientesdb.dump
```

```
$ docker compose exec pgdb pg_dump -U admin -d clientesdb -Fc -f /tmp/clientesdb.dump
$ docker compose exec pgdb pg_dump -U admin -d clientesdb -f /tmp/clientesdb.sql
$ docker compose exec pgdb ls -lh /tmp/clientesdb.dump /tmp/clientesdb.sql
-rw-r--r-- 1 root root 5.0K Nov  4 10:41 /tmp/clientesdb.dump
-rw-r--r-- 1 root root 3.6K Nov  4 10:41 /tmp/clientesdb.sql
$ docker compose exec pgdb pg_restore -l /tmp/clientesdb.dump
;
; Archive created at 2026-11-04 10:41:46 UTC
;     dbname: clientesdb
;     TOC Entries: 19
;     Compression: gzip
;     Dump Version: 1.15-0
;     Format: CUSTOM
;     Integer: 4 bytes
;     Offset: 8 bytes
;     Dumped from database version: 16.3 (Debian 16.3-1.pgdg120+1)
;     Dumped by pg_dump version: 16.3 (Debian 16.3-1.pgdg120+1)
;
;
; Selected TOC Entries:
;
216; 1259 16386 TABLE public clientes admin
215; 1259 16385 SEQUENCE public clientes_id_seq admin
3368; 0 0 SEQUENCE OWNED BY public clientes_id_seq admin
218; 1259 16393 TABLE public pedidos admin
217; 1259 16392 SEQUENCE public pedidos_id_seq admin
3369; 0 0 SEQUENCE OWNED BY public pedidos_id_seq admin
3208; 2604 16389 DEFAULT public clientes id admin
3209; 2604 16396 DEFAULT public pedidos id admin
3359; 0 16386 TABLE DATA public clientes admin
3361; 0 16393 TABLE DATA public pedidos admin
3370; 0 0 SEQUENCE SET public clientes_id_seq admin
3371; 0 0 SEQUENCE SET public pedidos_id_seq admin
3211; 2606 16391 CONSTRAINT public clientes clientes_pkey admin
3213; 2606 16398 CONSTRAINT public pedidos pedidos_pkey admin
3214; 2606 16399 FK CONSTRAINT public pedidos pedidos_cliente_id_fkey admin
$ docker compose cp pgdb:/tmp/clientesdb.dump ./clientesdb.dump
 src-pgdb-1 Copying src-pgdb-1:/tmp/clientesdb.dump to ./clientesdb.dump
 src-pgdb-1 Copied src-pgdb-1:/tmp/clientesdb.dump to ./clientesdb.dump
```

Copia de seguridad en los dos formatos, tamaños y contenido del índice

Paso 3. Restauración en una base de datos nueva

Es la forma más segura de comprobar una copia, porque no toca los datos originales. Se crea una base de datos vacía y se restaura en ella:

```
docker compose exec pgdb createdb -U admin clientesdb_restaurada
docker compose exec pgdb pg_restore -U admin -d clientesdb_restaurada /tmp/clientesdb.dump
docker compose exec pgdb psql -U admin -d clientesdb_restaurada -c "SELECT 'clientes' AS tabla, count(*),
        md5(string_agg(c::text, ',' ORDER BY id)) FROM clientes c UNION ALL SELECT 'pedidos', count(*),
        md5(string_agg(p::text, ',' ORDER BY id)) FROM pedidos p;"
```

El recuento y los MD5 coinciden con los del paso 1.

```
$ docker compose exec pgdb createdb -U admin clientesdb_restaurada
$ docker compose exec pgdb pg_restore -U admin -d clientesdb_restaurada /tmp/clientesdb.dump
$ docker compose exec pgdb psql -U admin -d clientesdb_restaurada -c "SELECT 'clientes' AS tabla, count(*),
    md5(string_agg(c::text, ',' ORDER BY id)) FROM clientes c UNION ALL SELECT 'pedidos', count(*),
    md5(string_agg(p::text, ',' ORDER BY id)) FROM pedidos p;"
  tabla   | count |               md5
----------+-------+----------------------------------
 clientes |     3 | 6fa9339d883070e9a704739cd835dcee
 pedidos  |     3 | 0126617ce5584df79682981be43f6da7
(2 rows)
```

Restauración en clientesdb_restaurada y verificación

Paso 4. Desastre simulado y restauración sobrescribiendo

Se simula un error humano grave: el borrado accidental de las tablas. Se comprueba que los datos han desaparecido y se recuperan desde la copia con `--clean --if-exists`, que elimina los objetos existentes antes de recrearlos (y no falla si ya no existen):

```
docker compose exec pgdb psql -U admin -d clientesdb -c "DROP TABLE pedidos; DROP TABLE clientes;"
docker compose exec pgdb psql -U admin -d clientesdb -c "\dt"
docker compose exec pgdb pg_restore -U admin -d clientesdb --clean --if-exists /tmp/clientesdb.dump
docker compose exec pgdb psql -U admin -d clientesdb -c "SELECT 'clientes' AS tabla, count(*),
        md5(string_agg(c::text, ',' ORDER BY id)) FROM clientes c UNION ALL SELECT 'pedidos', count(*),
        md5(string_agg(p::text, ',' ORDER BY id)) FROM pedidos p;"
```

Los MD5 vuelven a coincidir con los del paso 1: la recuperación ha sido completa, incluyendo la clave foránea de `pedidos` hacia `clientes`.

```
$ docker compose exec pgdb psql -U admin -d clientesdb -c "DROP TABLE pedidos; DROP TABLE clientes;"
DROP TABLE
DROP TABLE
$ docker compose exec pgdb psql -U admin -d clientesdb -c "\dt"
Did not find any relations.
$ docker compose exec pgdb pg_restore -U admin -d clientesdb --clean --if-exists /tmp/clientesdb.dump
$ docker compose exec pgdb psql -U admin -d clientesdb -c "SELECT 'clientes' AS tabla, count(*),
    md5(string_agg(c::text, ',' ORDER BY id)) FROM clientes c UNION ALL SELECT 'pedidos', count(*),
    md5(string_agg(p::text, ',' ORDER BY id)) FROM pedidos p;"
  tabla   | count |               md5
----------+-------+----------------------------------
 clientes |     3 | 6fa9339d883070e9a704739cd835dcee
 pedidos  |     3 | 0126617ce5584df79682981be43f6da7
(2 rows)
```

Borrado accidental y recuperación con pg_restore --clean

Paso 5. Restauración de la copia SQL plana

La copia en texto no se puede cargar con `pg_restore`, sino con `psql`. Se restaura en otra base de datos nueva:

```
docker compose exec pgdb createdb -U admin clientesdb_sql
docker compose exec pgdb psql -U admin -d clientesdb_sql -f /tmp/clientesdb.sql
docker compose exec pgdb psql -U admin -d clientesdb_sql -c "SELECT 'clientes' AS tabla, count(*),
        md5(string_agg(c::text, ',' ORDER BY id)) FROM clientes c UNION ALL SELECT 'pedidos', count(*),
        md5(string_agg(p::text, ',' ORDER BY id)) FROM pedidos p;"
```

```
$ docker compose exec pgdb createdb -U admin clientesdb_sql
$ docker compose exec pgdb psql -U admin -d clientesdb_sql -f /tmp/clientesdb.sql
[... 16 líneas omitidas ...]
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
ALTER TABLE
ALTER TABLE
COPY 3
COPY 3
[... 10 líneas omitidas ...]
ALTER TABLE
ALTER TABLE
ALTER TABLE
$ docker compose exec pgdb psql -U admin -d clientesdb_sql -c "SELECT 'clientes' AS tabla, count(*),
    md5(string_agg(c::text, ',' ORDER BY id)) FROM clientes c UNION ALL SELECT 'pedidos', count(*),
    md5(string_agg(p::text, ',' ORDER BY id)) FROM pedidos p;"
  tabla   | count |               md5
----------+-------+----------------------------------
 clientes |     3 | 6fa9339d883070e9a704739cd835dcee
 pedidos  |     3 | 0126617ce5584df79682981be43f6da7
(2 rows)
```

Restauración de la copia SQL con psql

3. Equivalencia con pgAdmin y conclusiones

Acción en pgAdmin	Herramienta equivalente	Uso
Backup… (formato Custom)	`pg_dump -Fc`	Copia comprimida y restaurable de forma selectiva
Backup… (formato Plain)	`pg_dump`	Copia en texto SQL, cargable con `psql`
Restore…	`pg_restore`	Restaura una copia Custom/Tar en una BD existente

	Formato personalizado (`-Fc`)	SQL plano
Se restaura con	`pg_restore`	`psql`
Tamaño	Comprimido	Mayor
Restauración selectiva y paralela	Sí	No
Legible y editable a mano	No	Sí

Conclusiones. (1) Una copia solo sirve si se ha probado a restaurarla: por eso se verifica con MD5 en cada restauración y se ha comprobado el índice con `pg_restore -l`. (2) Restaurar primero en una base de datos nueva evita riesgos; `--clean --if-exists` es la opción cuando se quiere sobrescribir. (3) Estas son copias lógicas: recuperan el estado en el momento de la copia, pero no permiten volver a un instante intermedio. Para eso existen la copia física (`pg_basebackup`) y el archivado de WAL, que permiten la recuperación en un punto en el tiempo (PITR) y no se han utilizado en esta práctica. (4) Una copia almacenada en el mismo equipo que la base de datos no protege frente a la pérdida de ese equipo, por lo que se copió al anfitrión.